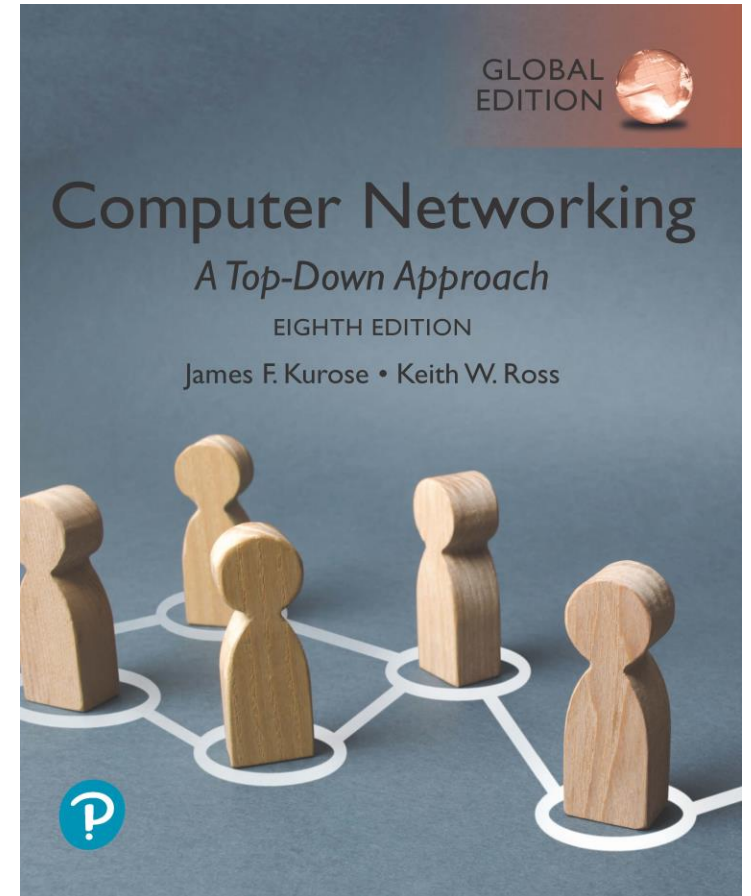


Chapter 3

Transport Layer



Computer Networking: A Top-Down Approach

8th Edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- **TCP congestion control**
- Evolution of transport-layer functionality



TCP phases

All TCP flows have a connection phase and transfer phase. The connection phase involves the standard three way handshake between sender and receiver with SYN/SYN+ACK messages. After completion of this phase, the flow enters into the transfer phase.

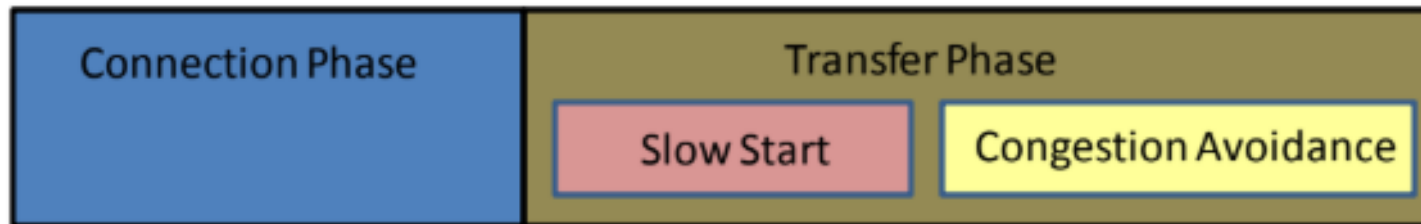
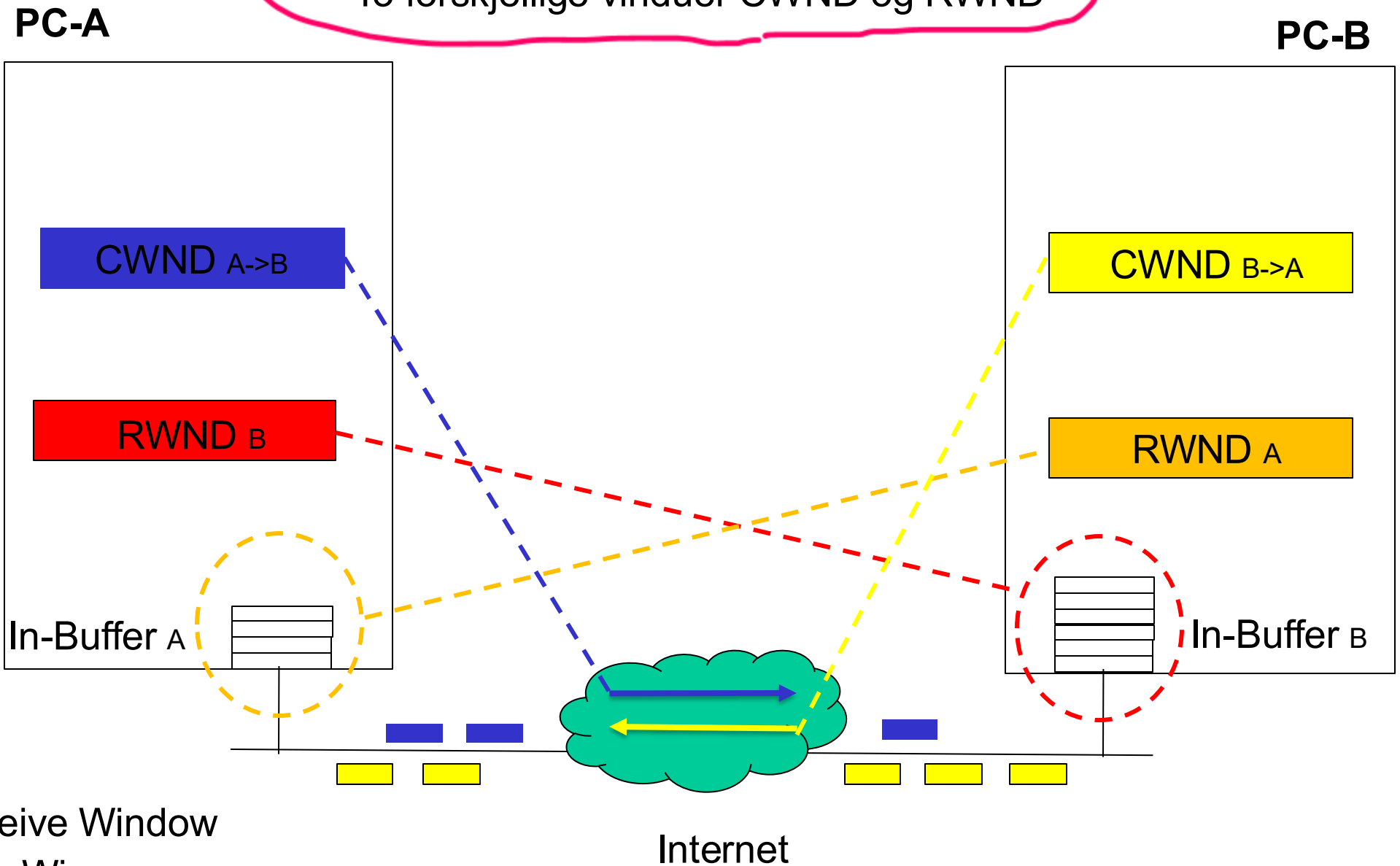


Figure 19 TCP flow phases

The transfer phase consists of a Slow Start (SS) and a Congestion Avoidance (CA) period. It should be acknowledged that SS and CA periods were not part of the initial TCP specification as per [40], but introduced by TCP Tahoe [23] and then later adopted by the majority of other versions.

To PC'er med TCP trafikk begge veier
To forskjellige vinduer CWND og RWND



RWND=Receive Window
CWND=CongWin

All TCP flows have a connection phase and transfer phase. The connection phase involves the standard three way handshake between sender and receiver with SYN/SYN+ACK messages. After completion of this phase, the flow enters into the transfer phase.

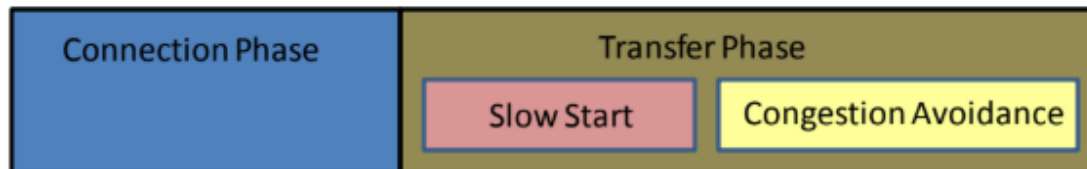


Figure 19 TCP flow phases

[23] JACOBSON, V. Congestion avoidance and control. *SIGCOMM Comput. Commun. Rev.* 18 (August 1988), 314–329.

[40] POSTEL, J. Transmission control protocol. RFC 793 (Standard), Sept. 1981. Updated by RFCs 1122, 3168, 6093.

The transfer phase consists of a Slow Start (SS) and a Congestion Avoidance (CA) period. It should be acknowledged that SS and CA periods were not part of the initial TCP specification as per [40], but introduced by TCP Tahoe [23] and then later adopted by the majority of other versions.

Merk: litt avhengig av kontekst så brukes *rwnd* og *cwnd* både som en variabel for å beskrive antall segmenter og antall bytes

Kan være litt forvirrende....

$$RWND = \frac{TCP\ Window}{MSS}$$

For a TCP flow i operating without congestion control and avoidance, i.e. pre-Tahoe, the resulting bandwidth in bytes/sec for this flow can then be expressed in the following way

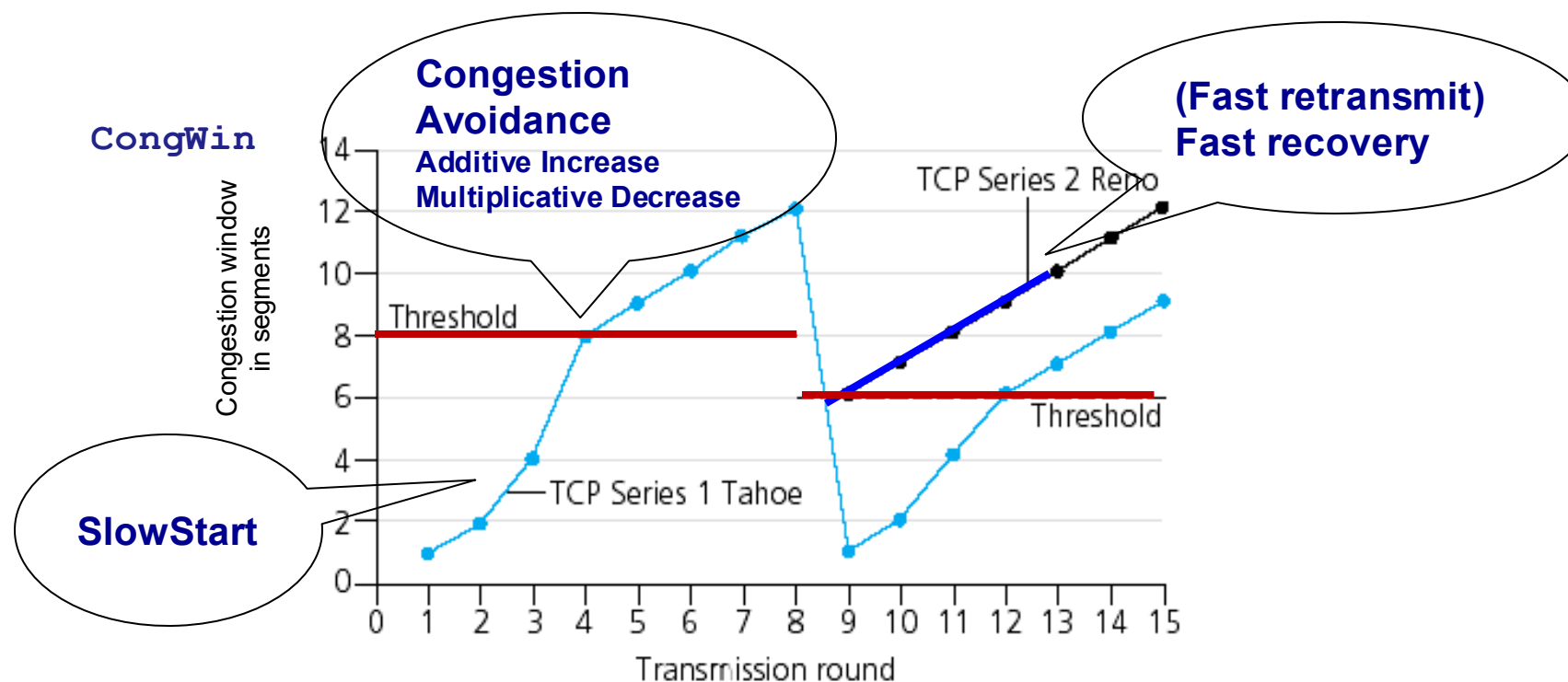
$$BW_i = \frac{TCP\ Window}{RTT} = \frac{RWND * MSS}{RTT}$$

Adding the congestion control and avoidance mechanism, the effective bandwidth for the same flow can then be expressed in the following modified way.

$$BW_i = \frac{TCP\ Window}{RTT} = \frac{\min(CWND, RWND) * MSS}{RTT}$$

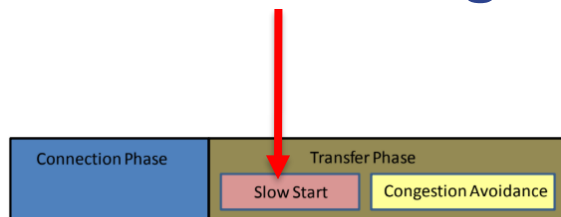
The three mechanisms of TCP congestion control

- When **CongWin** gets to 1/2 of its value before timeout: change from **exponential** to **linear** increase
- Implementation: variable **Threshold**
- At loss event, **Threshold** is set to 1/2 of **CongWin** just before loss event

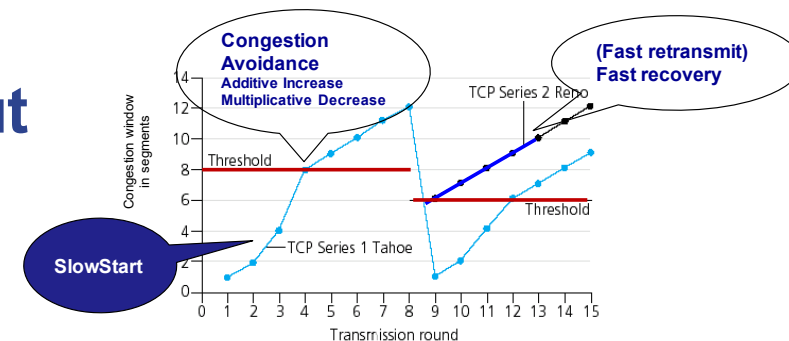
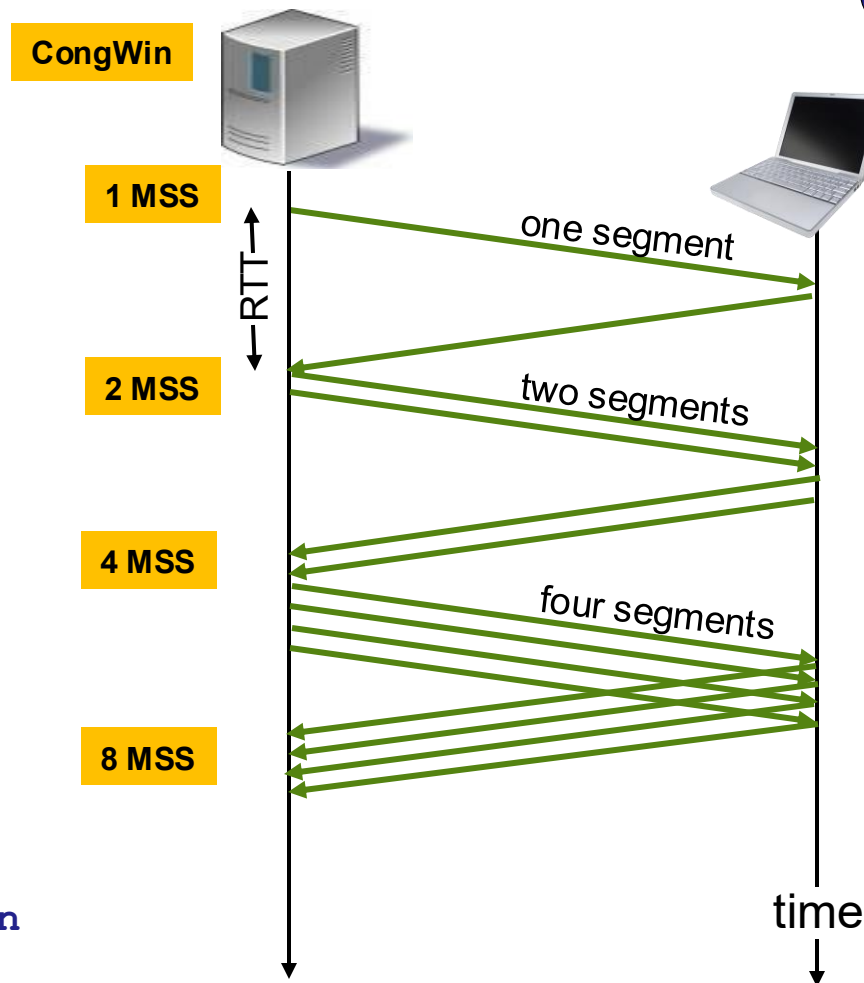


1 transmission round=RTT

TCP **slow start**: after connection set-up and timeout increase sending rate **exponentially**

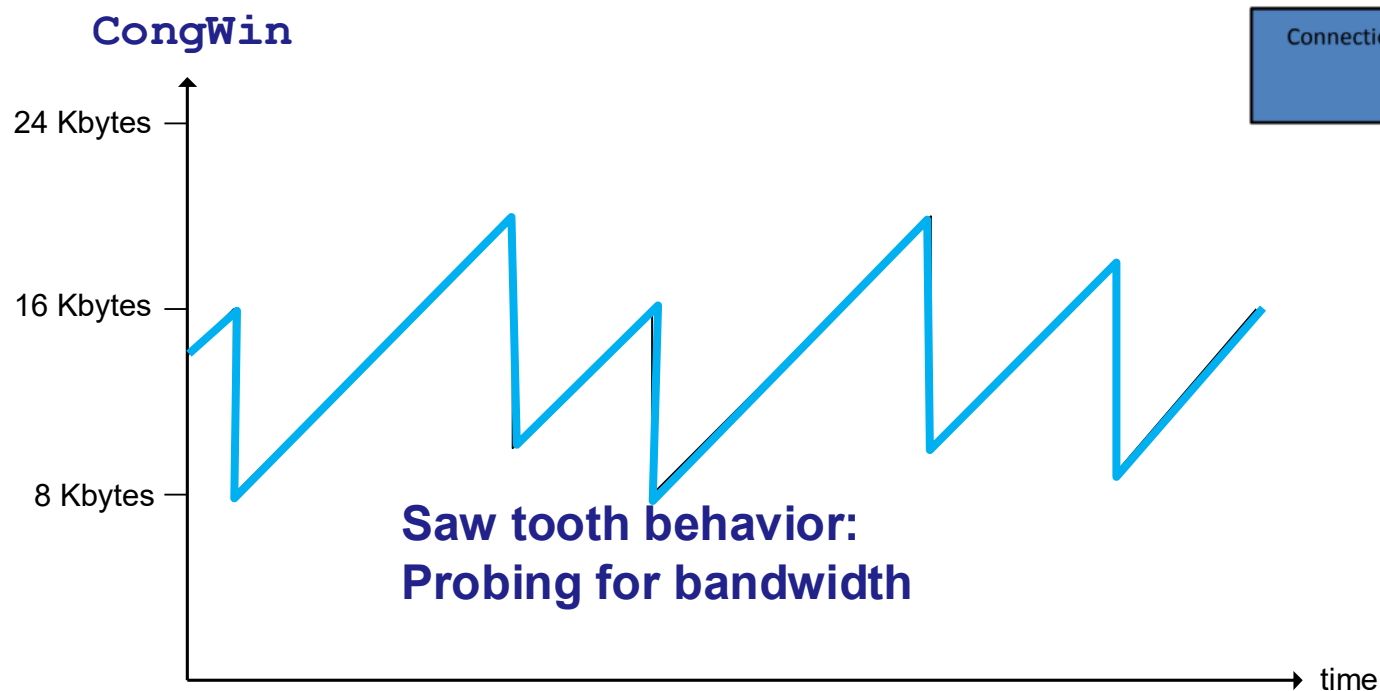
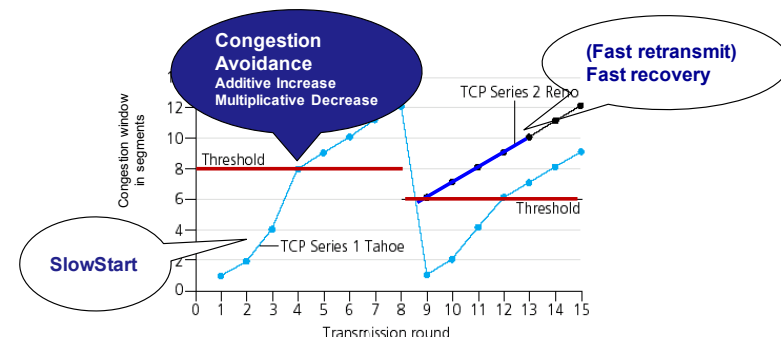


- At start of connection
CongWin = 1 MSS
 - If MSS = 500 bytes & RTT = 200 msec
initial rate = $\text{MSS}/\text{RTT} = 20 \text{ kbps}$
- Available bandwidth may be $\gg \text{MSS}/\text{RTT}$
 - desirable to quickly ramp up to respectable rate
 - double CongWin every RTT**
 - done by incrementing CongWin for every ACK received**



TCP congestion avoidance: additive increase, multiplicative decrease

- Increase transmission rate, **probing for usable bandwidth, until loss occurs**
 - **additive increase:** increase CongWin **by 1 MSS (maximum segment size) every RTT** until loss detected
 - **multiplicative decrease:** cut CongWin in half after loss



RTT vs CWND – dere må tenke **CWND** utvikling per RTT

Slow Start

$cwnd = cwnd + MSS$

(for hver ACK received)
(gir **eksp** vekst per RTT)

Eks Slow Start

Om cwnd var 4500 og MSS var 1500

Da kommer det 3 ACK i denne RTT

Ack1 → $cwnd = 4500 + 1500 = 6000$

Ack2 → $cwnd = 6000 + 1500 = 7500$

Ack3 → $cwnd = 7500 + 1500 = 9000$

Dvs at cwnd ble **doblet** til neste RTT

(for hver ACK received)
(gir **eksp** vekst per RTT)

Congestion Avoidance

$Cwnd = cwnd0 + \frac{MSS}{cwnd0}$ hvor da cwnd0 er start

(for hver ACK received)
(gir **lineær** vekst per RTT)

Eks Congestion Avoidance

Om cwnd var 3000 og MSS var 1500

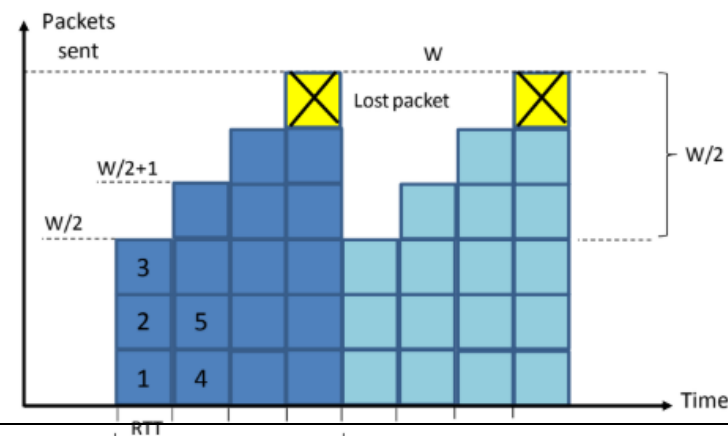
Da kommer det 2 ACK i denne RTT

Ack1 → $cwnd = 3000 + 1500(1500/3000) = 3750$

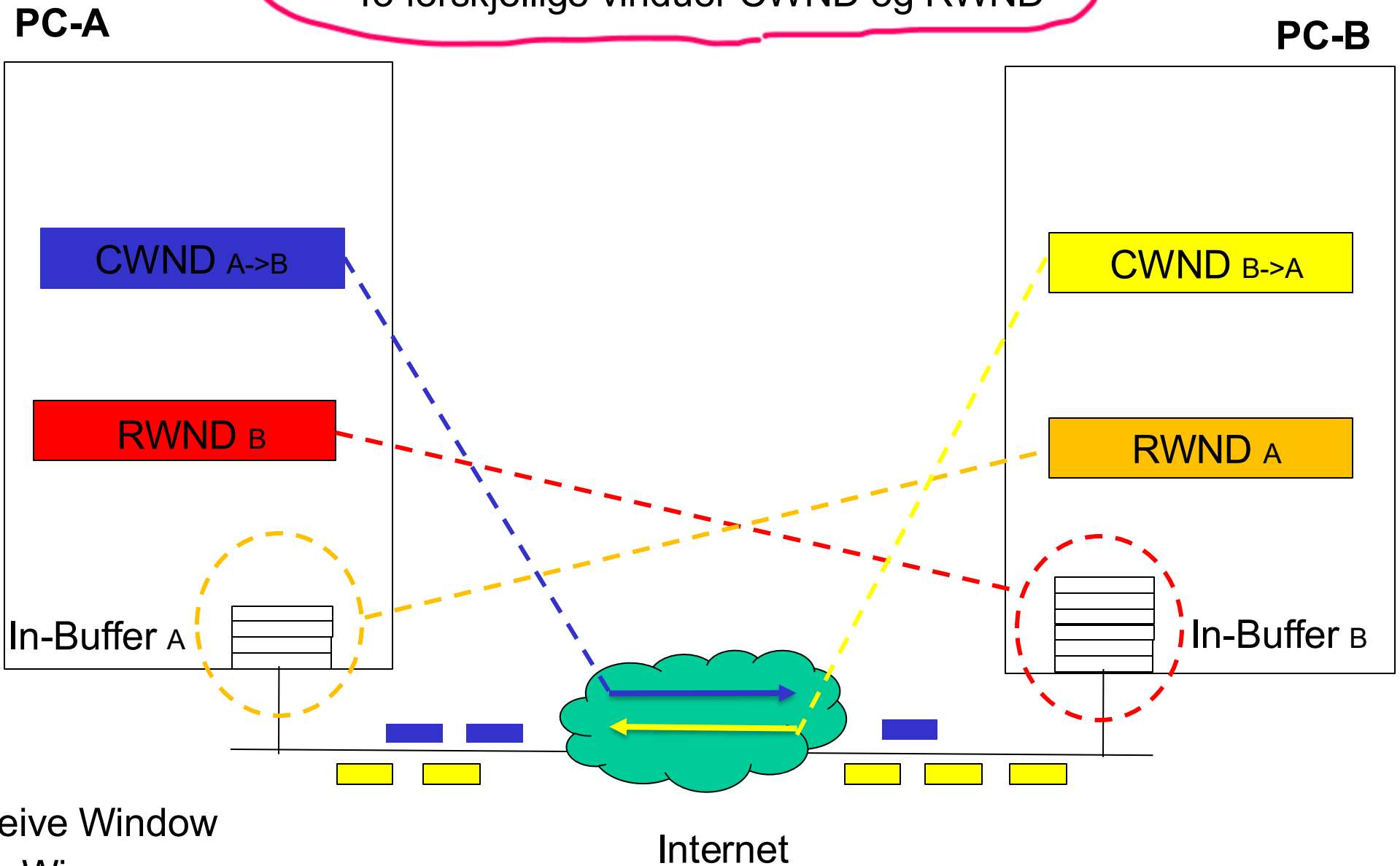
Ack2 → $cwnd = 3000 + 1500(1500/3000) + 1500(1500/3000) = 4500$

Dvs at cwnd ble **økt med en MSS** til neste RTT

Lineær vekst



To PC'er med TCP trafikk begge veier
To forskjellige vinduer CWND og RWND



RWND=Receive Window
CWND=CongWin

TCP-Hastighet: De to bremsene som styrer flyten

Å forklare hvordan samspillet mellom mottakerens kapasitet (rwnd) og nettverkets tilstand (cwnd) avgjør den faktiske overføringshastigheten i TCP.

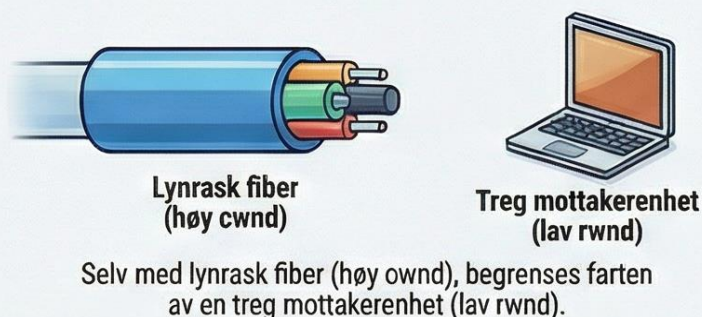


PRAKTISKE SCENARIER OG ANALOGI

Når nettverket er flaskehalsen



Når mottakeren er flaskehalsen



Lageret og Lastebilene



SAMMENLIGNING

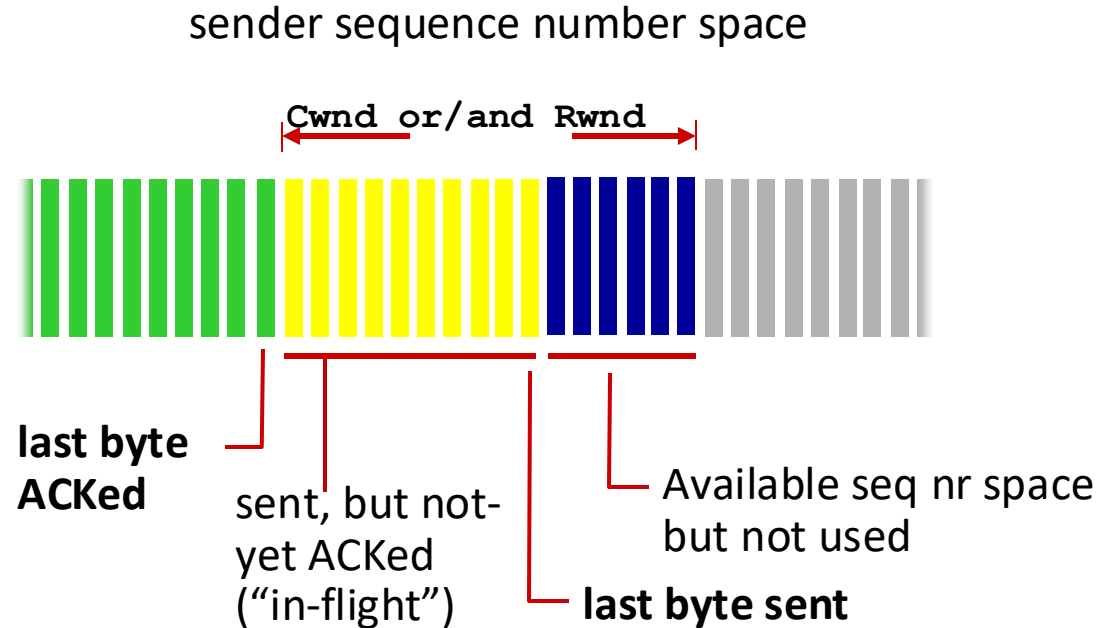
Receive Window (rwnd)

- Hvem bestemmer?: Mottakeren
- Formål: Flytkontroll
- Synlighet: Sendes i TCP-headeren

Congestion Window (cwnd)

- Hvem bestemmer?: Senderen (via algoritmer)
- Formål: Køkontroll
- Synlighet: Intern variabel (skjult)

TCP congestion control: details



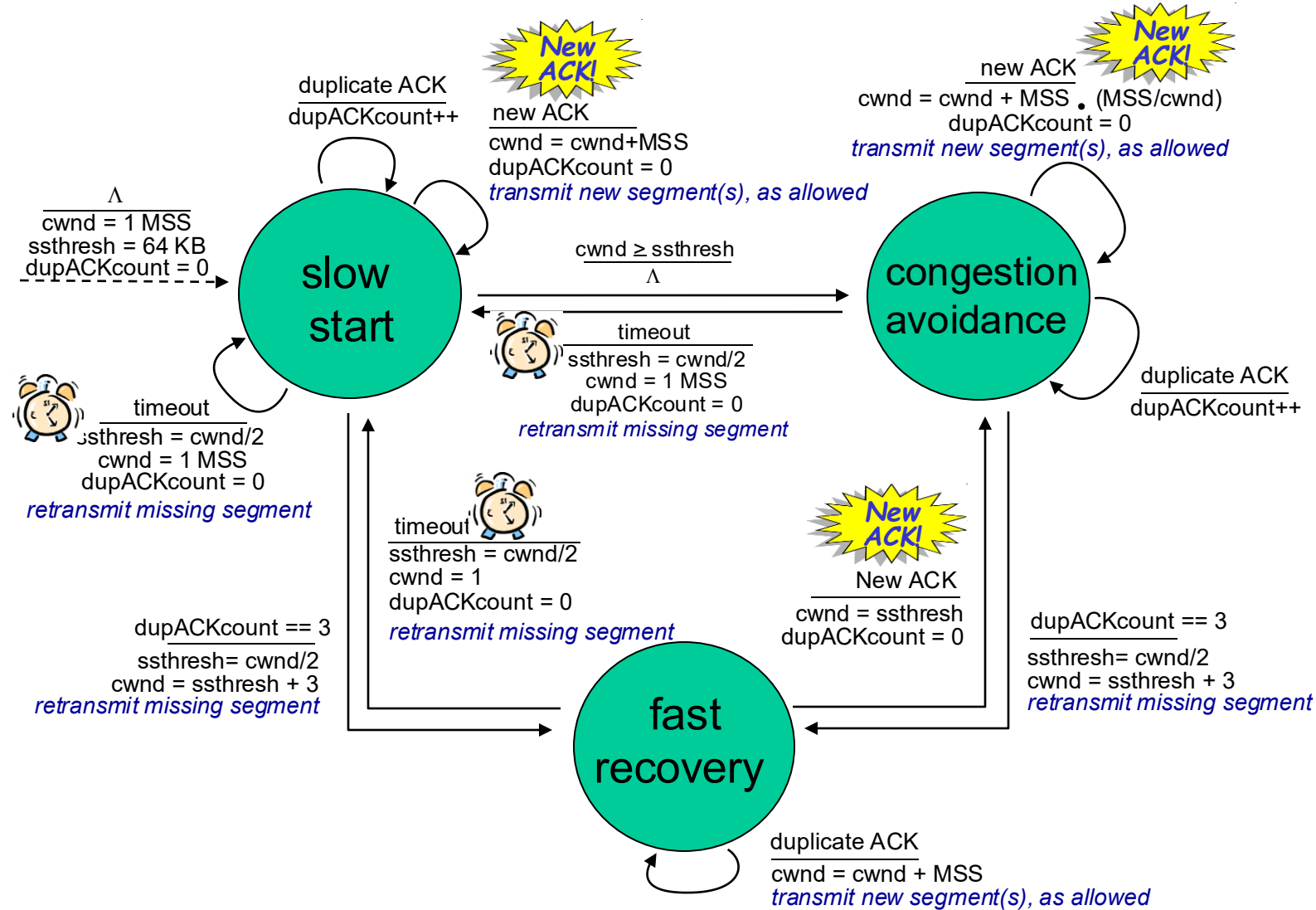
TCP sending behavior (when $cwnd < rwnd$):

- *roughly*: send $cwnd$ bytes, wait RTT for ACKS, then

$$\text{TCP rate} \approx \frac{cwnd}{RTT} \text{ bytes/sec}$$

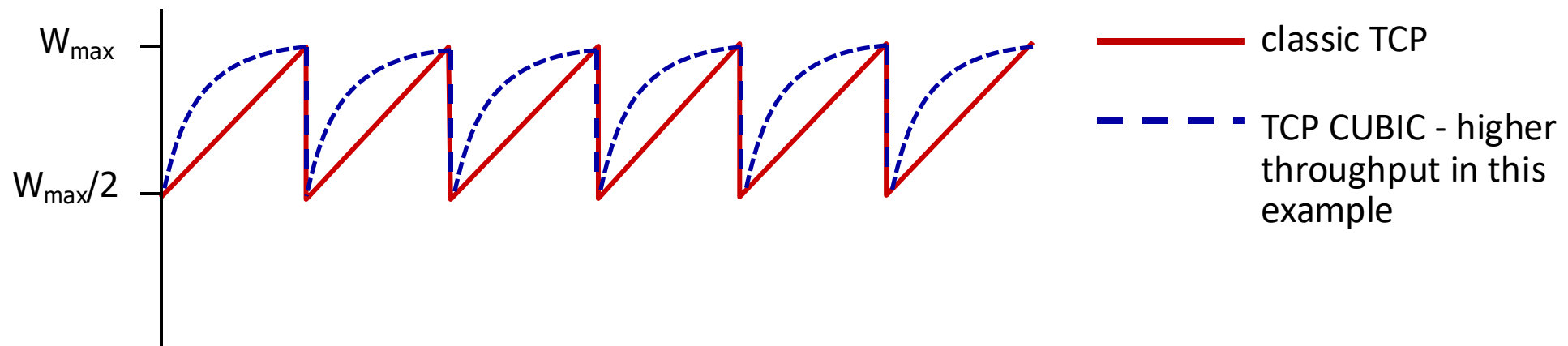
- TCP sender limits transmission: $\text{LastByteSent} - \text{LastByteAcked} \leq \text{Min}(cwnd, rwnd)$
- $cwnd$ is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)
- Note: $cwnd$ is "invisible", while $rwnd$ can be observed (value in header)

Summary: TCP congestion control



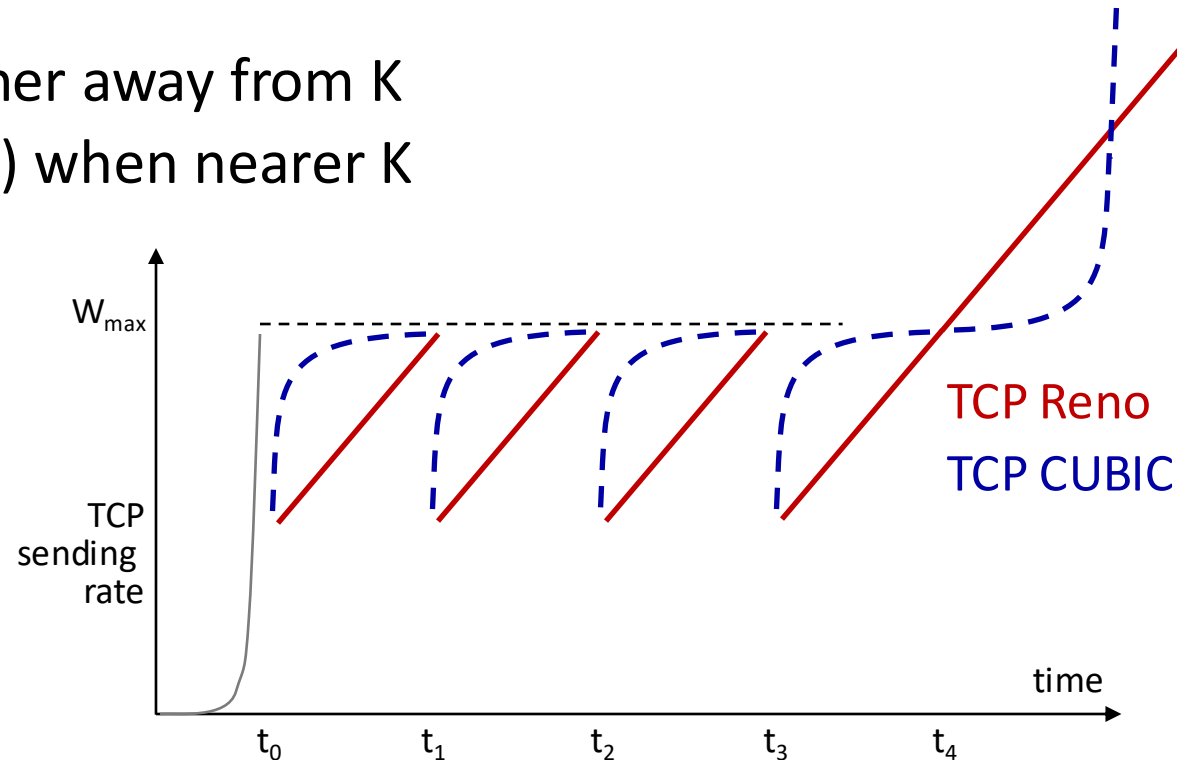
TCP CUBIC (a special version of TCP)

- Is there a better way than AIMD to “probe” for usable bandwidth?
- Insight/intuition:
 - $W_{\max}$: sending rate at which congestion loss was detected
 - congestion state of bottleneck link probably (?) hasn't changed much
 - after cutting rate/window in half on loss, initially ramp to to $W_{\max}$ *faster*, but then approach $W_{\max}$ more *slowly*



TCP CUBIC

- K: point in time when TCP window size will reach $W_{\max}$
 - K itself is tuneable
- increase W as a function of the *cube* of the distance between current time and K
 - larger increases when further away from K
 - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers



TCP CUBIC: Slik styres trafikken på det moderne internettet

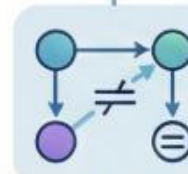
Standardalgoritmen for metningskontroll i moderne høyhastighetsnettverk, som bruker en matematisk kubisk funksjon for effektiv og stabil dataflyt.

Kjerneprinsipper for effektivitet



Kobler vekst fra rundturstid (RTT)

Vekst i overferingsvindu basert på tid siden forrige pakketap, uavhengig av avstand.



Rettferdighet på tvers av avstander

Sikrer at brukere nær serveren ikke dominerer båndbredden over de lenger unna.



TCP-vennlig modus

Bytter automatisk til standard TCP-oppførsel hvis mer effektivt i trege nettverk.

Fase 1: Hurtig gjenoppretting

Etter et pakketap økes farten raskt for å nå tilbake til forrige stabile nivå.



Fase 2: Stabilitetsplatået

Veksten flater ut nær forrige metningspunkt for å unngå nye sammenbrudd i nettverket.



Fase 3: Maksimal sondering

Hvis nettverket forblir stabilt, økes farten aggressivt for å finne ny tilgjengelig båndbredde.

Veksten flater ut nær forrige metningspunkt for å unngå nye sammenbrudd i nettverket.

Hvis nettverket forblir stabilt, økes farten aggressivt for å finne ny tilgjengelig båndbredde.

Nøkkelfunksjoner i Arkitekturen



HyStart

Sikker avslutning av oppstartsfasen for å unngå massive pakketap tidlig.



Veggklokke-tid

Bruker sanntid i stedet for ACK-responstid for å styre vindussterreisen.



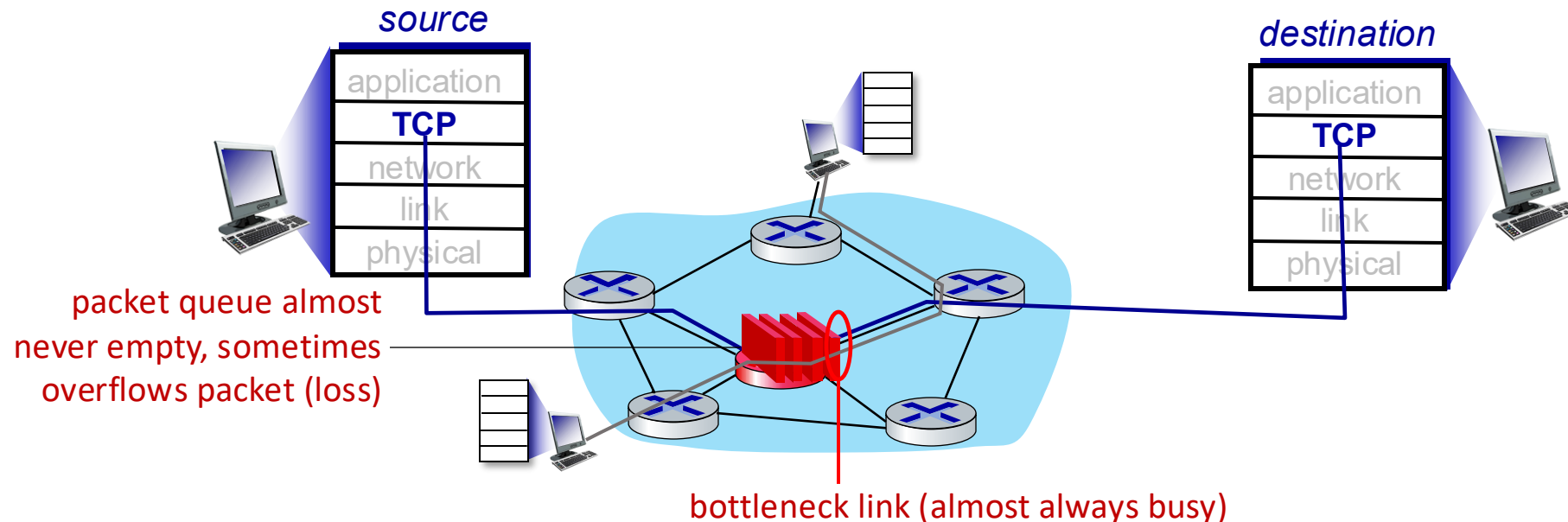
Kubisk funksjon

Den matematiske kjernen som dikterer den unike kurven for hastighetsvekst.

Transport Layer: 3-16

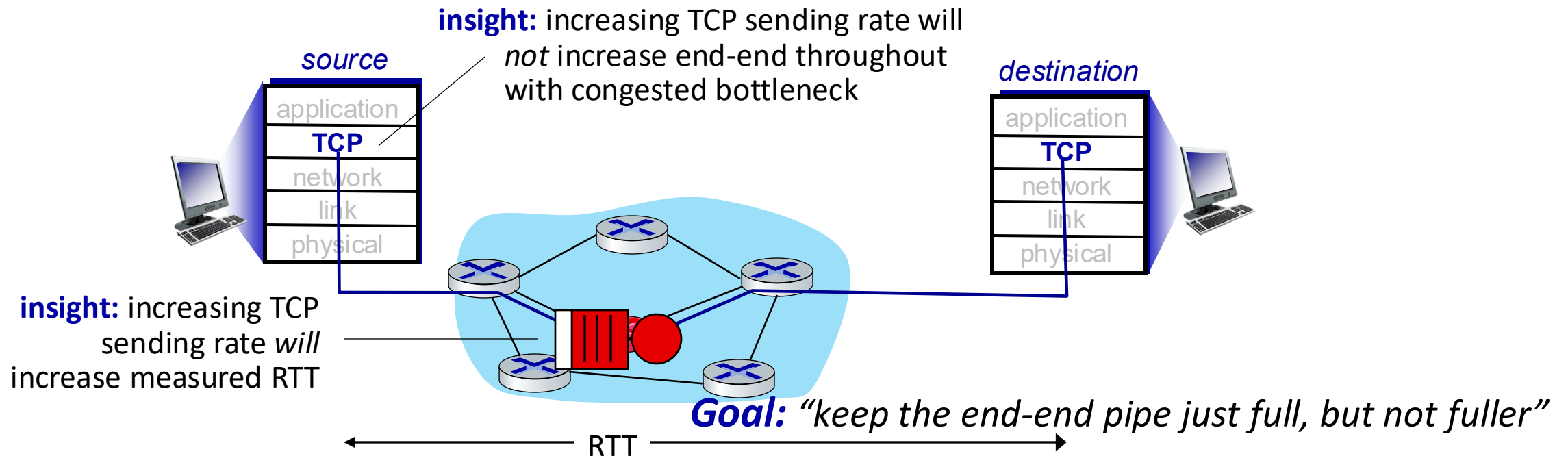
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*



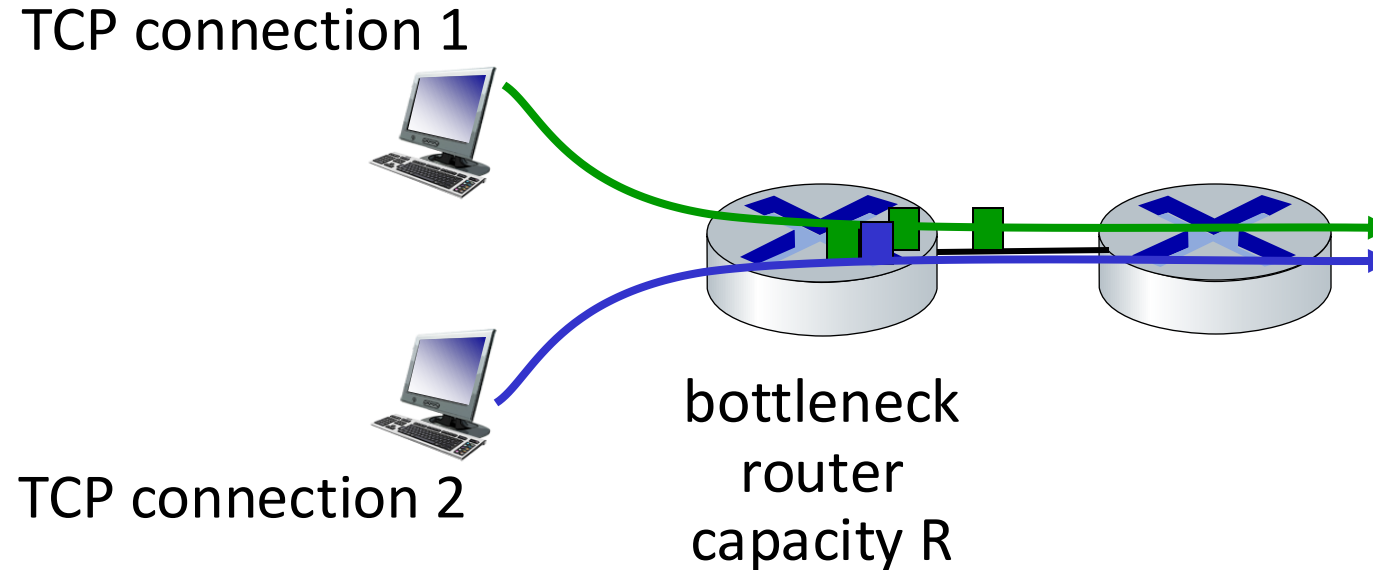
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link



TCP fairness

Fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K

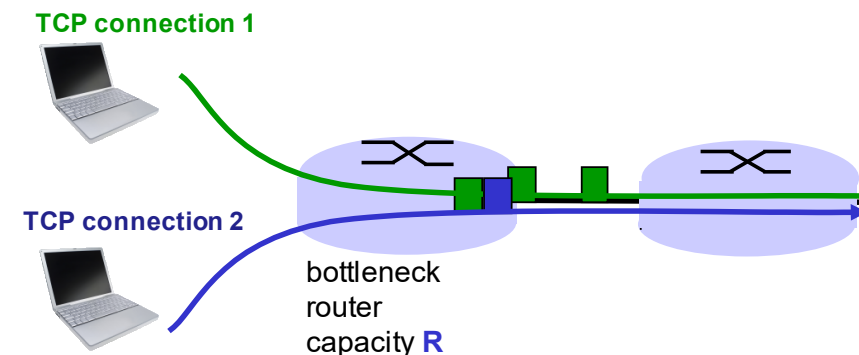
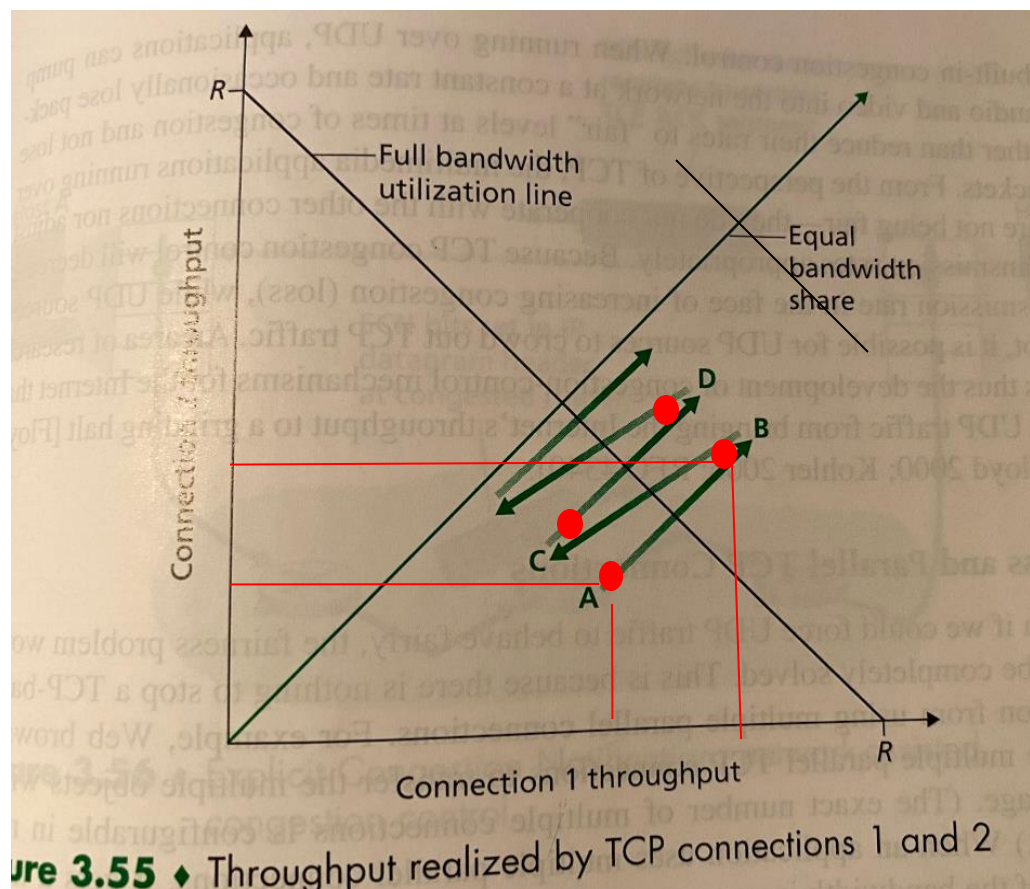


Why TCP fair – assuming AIMD

Additive Increase Multiplicative Decrease

Two competing sessions:

- Additive increase gives slope of 1, as throughput increases
- Multiplicative decrease decreases throughput proportionally



Fairness: each connection gets an equal share of available bandwidth

- A : utg pkt, total last lavere enn R
- A -> B : begge sesjoner additive increase
- B : begge sesjoner får pakketap
- C : begge sesjoner halvert CWND
- C -> D : begge sesjoner additive increase

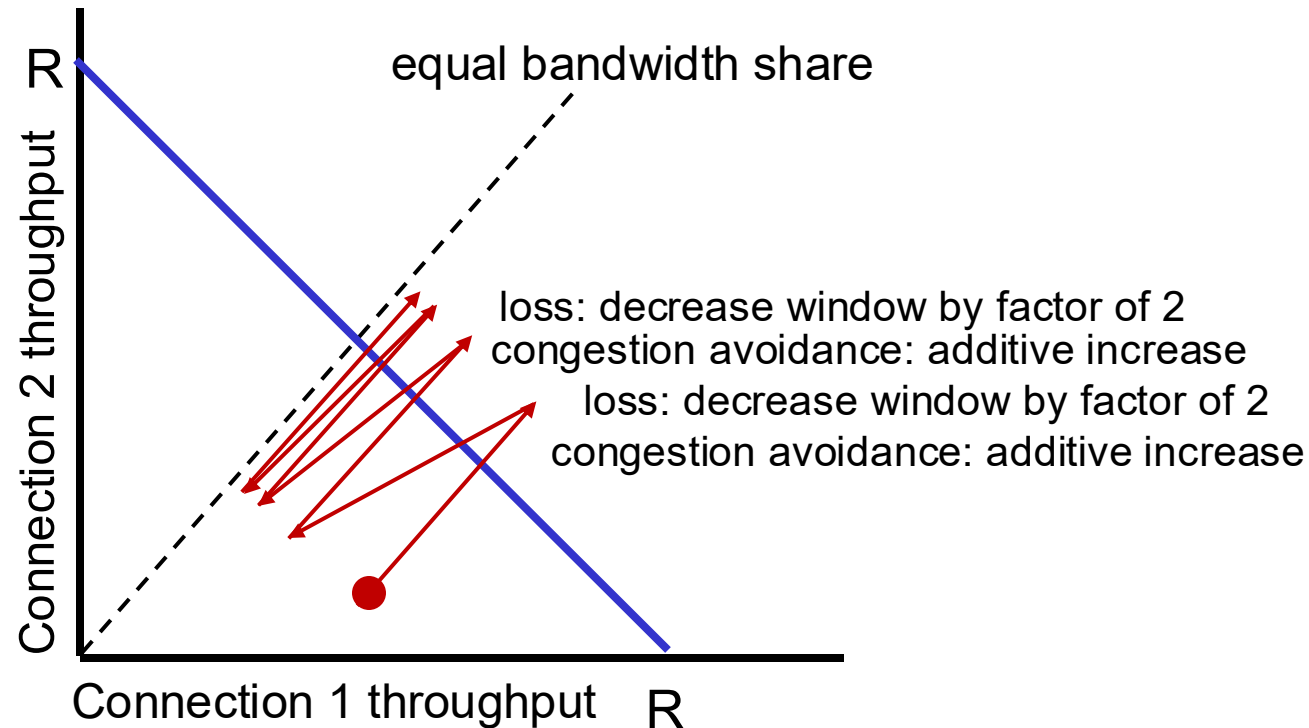
Konvergerer mot equal bw share....
convince yourself...

(veldig teoretisk case...)

Q: is TCP Fair?

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughput increases
- multiplicative decrease decreases throughput proportionally



Is TCP fair?

A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance

Fairness: must all network apps be “fair”?

Fairness and UDP

- multimedia apps often do not use TCP (**really...?**)
 - do not want rate throttled by congestion control
- instead use UDP:
 - send audio/video at constant rate, tolerate packet loss
- there is no “Internet police” policing use of congestion control

Fairness, parallel TCP connections

- application can open *multiple* parallel connections between two hosts
- web browsers do this , e.g., link of rate R with 9 existing connections:
 - new app asks for 1 TCP, gets rate $R/10$
 - new app asks for 11 TCPs, gets $R/2$